Problem Set 7

Foundations of Machine Learning

- **Submission**: Turn in as a `PDF` and a **separate source code file** on `https://mycourses.unh.edu`. Scanned handwritten notes are encouraged.
- **Questions**: See the class website for a link to office hours.

Relevant reading: MML 7.1, 7.3. The book is available online for free.

Problems marked **[code]** should be submitted as a program in a language of your choice (Julia or Python recommended).

1. Recall that one way of framing the projection of $\boldsymbol{b} \in \mathbb{R}^m$ onto the column space of $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, whose columns are linearly independent, is to see it as computing the point $\boldsymbol{p} \in \mathbb{R}^m$ closest to $\boldsymbol{b}$ (in the Euclidean norm) such that $\boldsymbol{p} = \boldsymbol{A}\boldsymbol{x}$ for some $\boldsymbol{x} \in \mathbb{R}^n$. In other words, $\boldsymbol{p}$ is a linear combination of columns of $\boldsymbol{A}$. Given this setup, please answer the following questions:
   (a) Formulate the projection described above as an optimization (minimization) problem.
   (b) Derive the gradient and the Hessian of the objective function.
   (c) Determine whether the optimization problem is guaranteed to be convex or not.
   (d) Derive the gradient descent update for this problem for some step size α.
   (e) Use the first-order optimality conditions to derive an optimal $\boldsymbol{x}$ and argue that this $\boldsymbol{x}$ is a global minimum of the objective. If there are any matrix inverses involved, argue why they exist. Is this solution preferable to using gradient descent?
   (f) Briefly show how you get the orthogonal projection matrix $\boldsymbol{A}(\boldsymbol{A}^\top \boldsymbol{A})^{-1}\boldsymbol{A}^\top$ from the derivations above.
2. **[code]** Solve the projection problem above using gradient descent for
$$\boldsymbol{A} = \begin{bmatrix} 1 & 5 \\ 1 & 12 \\ 1 & -1 \\ 1 & 6 \end{bmatrix}, \qquad \boldsymbol{b} = \begin{bmatrix} 7.2 \\ 13.8 \\ 0.1 \\ 10.2 \end{bmatrix}.$$
   Compare your solution with the closed-form solution from Problem 1(e). Report the step size and the number of iterations you used. Try several step sizes (such as, $\alpha \in \{10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$), plot the objective value against the iteration number, and describe what happens when α is too small or too large.
3. Suppose that you are given the following function $f \colon \mathbb{R}^n \to \mathbb{R}$ defined as
$$f(\boldsymbol{x}) = \sum_{i=1}^{m} \left(y_i \log\left(\frac{\exp(\boldsymbol{a}_i^\top \boldsymbol{x})}{1 + \exp(\boldsymbol{a}_i^\top \boldsymbol{x})} \right) + (1 - y_i) \log\left(\frac{1}{1 + \exp(\boldsymbol{a}_i^\top \boldsymbol{x})} \right) \right), \tag{1}$$
   where $y_i \in \{0, 1\}$ and $\boldsymbol{a}_i \in \mathbb{R}^n$ are some given values. Derive the gradient of this function.
4. **[code]** Verify the gradient of the function in (1) using an automatic differentiation tool (e.g., ForwardDiff.jl in Julia or JAX/PyTorch in Python). Generate random (or arbitrary) $\boldsymbol{a}_i$, y_i, and $\boldsymbol{x}$, and check that the gradient you derived in Problem 3 matches the one computed by automatic differentiation.
5. **Time spent.** Report approximately how many hours you spent completing this problem set (including the reading).